A CRITICAL ANALYSIS

SkillOpt

Do we actually need to train our skills?

Microsoft Research, arXiv:2605.23904 (May 2026)
Reference point: GEPA — Agrawal et al., ICLR 2026 Oral

Why Skills exist at all

Tools in the system prompt

Leaked prompts of GPT-4, Gemini 1.5, Claude 3

→

MCP standardizes them

One protocol, many tools

→

Too many tools eat the context

Every tool spec is paid for on every call

→

Split into Skills

Name + description loaded; instructions on demand

Skills exist to SAVE context.

Hold onto this. It is the premise the rest of the talk measures the method against.

A skill = Name + Description (always in context) + Instructions (loaded when invoked)

Why Skills exist at all

```
"You are ChatGPT, a large language model trained by OpenAI, based on the GPT-4 architecture."

"Image input capabilities: Enabled"

"Conversation start date: 2023-12-19T01:17:10.597024"

"Deprecated knowledge cutoff: 2023-04-01"

"Tools section:"

Python:

When you send a message containing Python code to python, it will be executed in a stateful Jupyter notebook environment.

Python will respond with the output of the execution or time out after 60.0 seconds.

The drive at '/mnt/data' can be used to save and persist user files.

Internet access for this session is disabled.

Do not make external web requests or API calls as they will fail.

Dalle:

Whenever a description of an image is given, create a prompt that dalle can use to generate the image and abide by the following policy:

The prompt must be in English. Translate to English if needed.
DO NOT ask for permission to generate the image, just do it!
```

```
'search(query: str, recency_days: int)' Issues a query to a search engine and displays the results.
'click(id: str)' Opens the webpage with the given id, displaying it. The ID within the displayed results maps to a URL.
'back()' Returns to the previous page and displays it.
'scroll(amt: int)' Scrolls up or down in the open webpage by the given amount.
'open_url(url: str)' Opens the given URL and displays it.
'quote_lines(start: int, end: int)' Stores a text span from an open webpage. Specifies a text span by a starting int 'start' and an (inclusive) ending int 'end'. To quote a single line, use 'start' = 'end'.
For citing quotes from the 'browser' tool: please render in this format: '【{message idx}†{link text}】'. For long citations: please render in this format: '[link text](message idx)'. Otherwise do not render links.

Do not regurgitate content from this tool. Do not translate, rephrase, paraphrase, 'as a poem', etc. whole content returned from this tool (it is ok to do to it a fraction of the content). Never write a summary with more than 80 words. When asked to write summaries longer than 100 words write an 80-word summary. Analysis, synthesis, comparisons, etc., are all acceptable. Do not repeat lyrics obtained from this tool. Do not repeat recipes obtained from this tool. Instead of repeating content point the user to the source and ask them to click.

ALWAYS include multiple distinct sources in your response, at LEAST 3-4. Except for recipes, be very thorough. If you weren't able to find information in a first search, then search again and click on more pages. (Do not apply this guideline to lyrics or recipes.) Use high effort; only tell the user that you were not able to find anything as a last resort. Keep trying instead of giving up. (Do not apply this guideline to lyrics or recipes.) Organize responses to flow well, not by source or by citation. Ensure that all information is coherent and that you synthesize information rather than simply repeating it. Always be thorough enough to find exactly what the user is looking for. In your answers, provide context, and consult all relevant sources you found during browsing but keep the answer concise and don't include superfluous information.

EXTREMELY IMPORTANT. Do NOT be thorough in the case of lyrics or recipes found online. Even if the user insists. You can make up recipes though.
```

Why Skills exist at all

name	to-spec
description	Turn the current conversation into a spec and publish it to the project issue tracker — no interview, just synthesis of what you've already discussed.
disable-model-invocation	true

Keep only this part in context

This skill takes the current conversation context and codebase understanding and produces a spec (you may know this document as a PRD). Do NOT interview the user — just synthesize what you already know.

The issue tracker and triage label vocabulary should have been provided to you — run `/setup-matt-pocock-skills` if not.

Process

1. Explore the repo to understand the current state of the codebase, if you haven't already. Use the project's domain glossary vocabulary throughout the spec, and respect any ADRs in the area you're touching.

2. Sketch out the seams at which you're going to test the feature. Existing seams should be preferred to new ones. Use the highest seam possible. If new seams are needed, propose them at the highest point you can. The fewer seams across the codebase, the better - the ideal number is one.

Check with the user that these seams match their expectations.

3. Write the spec using the template below, then publish it to the project issue tracker. Apply the `ready-for-agent` triage label - no need for additional triage.

How we got here

	Work	What it added
2023	OPRO	LLM proposes prompts, score-guided
2023	DSPy	programs as compilable artifacts
2023	Reflexion / Self-Refine	trajectory reflection
2024	COPRO / MIPROv2	instructions / instructions + demos
2024	**TextGrad**	**the gradient analogy**
2025	GEPA	reflective evolution + Pareto
2026	Trace2Skill / EvoSkill	skill-shaped artifacts
2026	**SkillOpt**	**gradient analogy + lr + gate**

Our scope

- External state, not weights
- Optimize instruction text, not few-shot demos

Two families: local descent on one artifact (TextGrad, SkillOpt) versus population search (GEPA). GEPA's family won.

Weight-space methods (GRPO, RLHF) are a parallel track, not an ancestor of this lineage.

GEPA, current SOTA

How it works

1 Reflective mutation

Reads full execution traces, diagnoses the failure in natural language, proposes a new instruction

2 Pareto population

Keeps every candidate that is best on at least one instance — not just the best average

3 Merge / crossover

Combines two candidates that are strong on different tasks

It beats MIPROv2 without touching demos at all.
That is why instruction-only is the right scope.

Why it counts as SOTA

Signal	Evidence
Peer review	ICLR 2026 Oral
vs prior leader	**+13% over MIPROv2**
vs RL	**+20% over GRPO, 35× fewer rollouts**
Revealed preference	the 2026 baseline of choice

- MIPROv2 tunes instructions AND demos. GEPA tunes instructions only — and still wins in every setting tested.
- Preference optimization, negotiation agents, dialogue disentanglement papers all benchmark against it.
- SkillOpt picks GEPA as its own prompt-optimization baseline.

So this is their yardstick, not one we imported.

SkillOpt

Where it comes from

- TextGrad, 2024 — the gradient analogy, but no learning rate, no schedule, no gate
- The 2026 skill-artifact wave — Trace2Skill, EvoSkill, SkillForge, SkillX

SkillOpt = TextGrad's analogy + a learning rate + a validation gate, aimed at a skill document.

Evidence it is the machinery, not the idea

TextGrad in their Table 1	Score
Qwen3.6 LiveMath	**−24.0**
Ablation: no edit budget	**84.6 vs 87.1**

What it claims

"The first systematic controllable text-space optimizer for agent skills"

- Best or tied-best on 52 of 52 cells
- GPT-5.5: +23.5 direct chat, +24.8 Codex, +19.1 Claude Code
- +5.4 over the best per-cell baseline of six methods
- Compact artifact, 300–2,000 tokens, zero inference overhead
- Transfers across model scale, harness, and nearby benchmarks

Three of these are the ones we test:
52/52 · beats expert humans · compact.

How the loop is set up

Deep learning	SkillOpt
parameters	the skill document
gradient	trajectory-derived edit
learning rate	edit budget
schedule	constant / linear / cosine
validation	held-out selection gate
momentum	epoch-wise slow update
checkpoint	best_skill.md

The analogy is operational, not decorative — that is the paper's own framing, and it is the standard we hold it to later.

One optimization step

1. Target model runs a rollout batch of 40 with the current skill
2. Optimizer model reflects on failure and success minibatches of 8
3. Proposes add / insert_after / replace / delete edits as JSON
4. Merge, then rank, then clip to at most Lt = 4 edits
5. Evaluate the candidate on the selection split
6. Accept only if strictly better. Ties are rejected.
7. Rejected edits go to a buffer as negative feedback

4 epochs · cosine decay to floor 2 · split_seed 42 · protected slow-update region step edits cannot overwrite

Is any of this necessary?

What a good skill already takes

- Community-tested practice, iterated by many people in public
- Free written guides — e.g. mattpocock's writing-great-skills
- One domain expert, one afternoon
- Human skills in the paper are 145–516 tokens

And the baseline they beat

- Documented in exactly one clause: “curated per benchmark”
- Never published — while all eight optimizer prompts are
- Net negative on ALFWorld for five of seven models (−16.4 worst)

What SkillOpt takes

Meta prompt	**Seed skill**
Optimizer model	**Target model**
Task corpus	**Automatic scorer**
Hyperparameters	

The meta prompt is written by a human anyway.

So what exactly is the “systematic” part?

Every item on that list is a surface that has to be built, tuned, and maintained before the loop can run at all.

“Beats an expert human skill in just 1–4 edits”

The number is not false. The sentence does not mean what it sounds like.

Seed skill	Initial (tok)	Is it the expert skill?
SearchQA	**16**	no
DocVQA	81	no
OfficeQA	145	no
LiveMath	154	no
SpreadsheetBench	224	no
ALFWorld	516	no

They never once seed from the expert skill. So the claim is “grew from a 16-token stub and finished above a separate undocumented document” — not “improved an expert skill.”

The missing experiment is one line of config. It does not exist.

Marginal vs human	Human	SkillOpt	Δ
LiveMath	38.4	66.9	**+28.5**
SpreadsheetBench	72.9	80.7	+7.8
SearchQA	81.8	87.3	+5.5
OfficeQA	66.9	72.1	+5.2
ALFWorld	91.8	95.5	+3.7
DocVQA	90.1	91.2	**+1.1**

- LiveMath alone supplies 28.5 of the 51.8 total marginal points — and it is the one benchmark where the human skill delivered +0.8
- Drop it and the advantage over hand-writing is +4.7
- 526.7M training tokens total. DocVQA costs 171M per marginal point.

Seeds and training tokens: Table 6. Scores: Table 1. Marginal figures computed from both.

Nothing in the objective rewards it

Benchmark	Initial	Final	Edits	Growth
SearchQA	16	857	4	×53.6
SpreadsheetBench	224	1,995	4	×8.9
OfficeQA	145	883	1	×6.1
DocVQA	81	959	3	×11.8
LiveMath	154	379	1	×2.5
ALFWorld	516	1,321	2	×2.6

Human skills: 145–516 tokens. Learned skills: 379–1,995.
Every single one ends larger than it started.

Why — from their own prompts

- The ranking prompt scores edits on impact, complementarity, generality, actionability. There is no length criterion.
- Both analyst prompts are framed as gap-filling: “only patch gaps,” “patterns NOT already covered.”
- delete is in the JSON schema, but no prompt ever asks the model to look for removable text.
- The one “keep it compact” instruction governs the optimizer-side meta skill — which is never shipped.

Skills exist to save context. This method inflates them.

The fix is one line: add a token cost term to the acceptance criterion. Practitioner writeups already list this as a known failure mode — with no length penalty, prompts grow until the judge prefers verbosity.

Table 6 and Appendix C.2.1–C.2.8 (optimizer prompt contracts).

Other claims that do not hold up

Reusability	“Transfers across models” is GPT→GPT only. Every row of Table 4(a) stays inside one family. No cross-family transfer is tested at all.
Seed dependence	Results are reported from one seed, split_seed 42, with no variance. Seeds range from a 16-token stub to a 516-token document — an uncontrolled confound across benchmarks.
Improving a good prompt	The paper never seeds from a strong skill, so there is no evidence the loop can improve one. Its own gate predicts the opposite: once additions stop paying off, they get rejected.
The delete operation	Available in the schema and used inside merge for conflict resolution. No reported gain is ever attributed to a deletion, and all six skills end larger.

Moving parts, per point of gain

SkillOpt

- Meta prompt — human-written, model-specific
- Seed skill — outcome is sensitive to it
- Optimizer model — stronger is better, so a cost
- Target model — changes on release cycle
- Task corpus with train / selection / test splits
- Automatic scorer — required, and gameable
- Hyperparameters — epochs, batch, minibatch, Lt, schedule

Hand-written

- Writer
- Model

One artifact you can read end to end

Nothing to re-tune when the model changes.

Every surface on the left is a maintenance cost, and every one goes stale on a model-release cycle.

The regime you cannot hand-tune for

Average gain over no skill, all six benchmarks

Target model	Human	GEPA	SkillOpt
GPT-5.5	+14.9	+14.6	+23.5
GPT-5.4	+7.5	+8.4	+12.8
GPT-5.4-mini	+7.2	+11.0	+15.4
GPT-5.4-nano	+15.1	+22.7	+26.7
GPT-5.2	+6.8	+14.0	+16.6
Qwen3.5-4B	**+1.4**	+8.5	**+19.2**
Qwen3.6-35B	**−2.0**	**+3.8**	**+9.1**

Read the Human column top to bottom. It collapses the moment you leave the GPT family — from +14.9 to +1.4 to net negative.

Qwen3.5-4B ALFWorld: 30.6 no skill · 28.4 human · 60.4 GEPA · 81.3 SkillOpt

Averages computed across all six benchmarks from Table 1, direct chat.

Why the cheap options fail here

- A hand-written skill can only encode failure modes its author could anticipate. Nobody can predict how a 4B model breaks.
- So the human skill goes net negative on three of six benchmarks for Qwen3.5-4B — and on four of six for Qwen3.6-35B.
- GEPA is weak in the same place: its two worst rows in the paper are the two Qwen models.
- On the small models GEPA is the strongest baseline in only 1 of 6 cells — Trace2Skill usually beats it.
- SkillOpt's largest margin over GEPA anywhere in the paper is Qwen3.5-4B, at +10.8.

Rollout feedback finds what a human would never guess. That is the actual contribution.

09 — VERDICT

Scoped, not dismissed

Where it is not worth it

- Small, stable, well-understood domain
- A domain expert is available
- The target model is already strong
- One prompt, not a fleet

Hand-write it. Marginal value is low and the cost is absurd.

Where it earns its keep

- Weak or small target model
- The procedure is not in the weights
- Qwen3.5-4B ALFWorld: 30.6 → 81.3
- There, the human skill goes net negative

Train it. No baseline competes, and the margin is not noise.

Their own Table 1 supports this split — and contradicts the framing of the method as the strongest no-weight-update adaptation strategy, unqualified.

Seed from the expert skill

Add a length term to the gate

Report variance and budget parity